

Trabajo Práctico Especial para finales LIBRES - válido para exámenes a rendir de Diciembre 2025 a Diciembre 2026

Defina las clases (nombre, superclase, atributos y métodos) para implementar una solución orientada a objetos para el siguiente problema e implemente en Java. Tener en cuenta todos los mecanismos de la programación Orientada a Objetos: Polimorfismo, abstracción, delegación, etc.

El *Juego de la Vida* es un "autómata celular" donde celdas en una cuadrícula (tablero) evolucionan de generación en generación basándose en reglas simples y el estado de sus vecinos.

El trabajo consiste en diseñar e implementar una solución Orientada a Objetos en Java que simule un tablero de $n \times m$ celdas que evolucionan con el tiempo. Cada celda en este tablero tendrá diferentes estados, y lo más importante es que el comportamiento de la celda cambiará dinámicamente según su estado actual. Inicialmente los posibles estados de la Celda son:

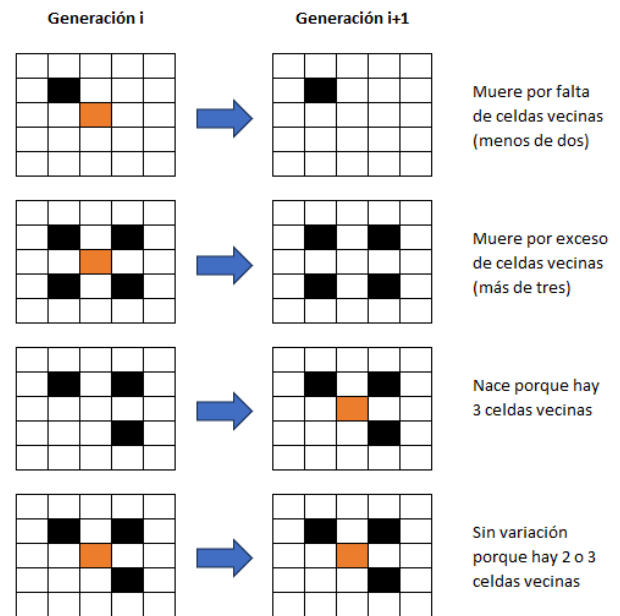
- Viva
- Muerta

Las reglas básicas del Juego de la Vida son las siguientes:

- Una celda Viva con menos de 2 vecinos vivos muere en la siguiente generación por soledad.
- Una celda Viva con 2 o 3 vecinos vivos sobrevive a la siguiente generación.
- Una celda Viva con más de 3 vecinos vivos muere en la siguiente generación por sobrepoblación.
- Una celda Muerta con exactamente 3 vecinos vivos se convierte en una celda "Viva" (reproducción) en la siguiente generación.

Se debe proveer una interfaz console/CLI obligatoria que permita:

1. Cargar un estado inicial desde archivo (ver formato abajo) o de manera aleatoria.
2. Ejecutar N generaciones o correr indefinidamente con intervalo configurable entre pasos.
3. Mostrar el estado de la grilla por pantalla en cada generación (ej.: . para muerto, **O** para vivo, **X** para latente).
4. Opcional (bonus): GUI usando Swing/JavaFX con visualización en tiempo real y controles (start/stop/step/speed):.



NOTA: Para pasar de una generación a la siguiente se siguen los siguientes pasos:

- Si no hubo cambios con respecto a la generación anterior en todo el tablero, el ciclo se corta.
- Se calcula el siguiente estado para cada celda del tablero, siguiendo las reglas básicas descritas anteriormente
- Tener en cuenta que para el cálculo del próximo estado de una celda, se tiene en cuenta los vecinos, pero no se debe actualizar el estado de una celda inmediatamente ya que el cálculo para todas las celdas se debe hacer con el estado actual.
- **Luego de que fueron calculados todos los nuevos estados, se debe actualizar el tablero con los datos de la nueva generación.**

Formato de archivo para estado inicial

Implementar al menos un formato simple (texto): primera línea con indicación de la cantidad de filas y columnas y luego, en cada fila caracteres que indiquen el estado de la celda: ej: X (vivo), . (muerto)

```
4 5
. . . . .
. . X . .
. . X . .
. . X . .
```

Extienda la solución anterior para agregar dos nuevos estados (**No se debe modificar ninguna clase de la solución anterior**):

- **Celda Enferma** (se considera viva). Una celda viva es factible que se enferme en la siguiente generación con una probabilidad del 25%. Si no se enferma evoluciona como se describió anteriormente. Una celda enferma en la siguiente generación morirá si o si.
- **Celda Latente** (se considera muerta). Una celda Latente con exactamente 1 vecino vivo se convierte en una celda "Viva" en la siguiente generación.

Pueden aparecer también otros estados. El soporte de inicialización debería considerar que se puedan utilizar cualquiera de los nuevos estados también.

Requisitos de diseño

- Separación clara entre lógica (modelo) y presentación (CLI/GUI).
- Manejo apropiado de errores (archivos inexistentes, formato inválido, parámetros inválidos).
- Permitir extender el juego (p. ej. otras reglas) con los mínimos cambios al código existente — documentar cómo se podría hacer.

Documentación y entrega

- README con instrucciones para compilar y ejecutar. Explicación del diseño (clases principales y responsabilidades).
- Al menos 3 archivos semilla incluidos en un directorio `ejemplos/`
- Código comentado y con convenios de nomenclatura consistentes.
- Enviar link a un repositorio GIT o el código del trabajo al mail de la cátedra con al menos 2 días de anticipación a la fecha del final que planea rendir

Final escrito en modalidad libre:

- Examen Práctico, es similar a un final regular de la materia aunque es posible que se agreguen items especiales para los que rindan en la modalidad libre
- Examen Teórico, se rinde junto con el examen Práctico (durante el mismo tiempo). En el mismo se realizan preguntas relativas a distintos conceptos de la materia.
- En caso de ser necesario, se puede concertar una entrevista posterior al examen escrito (fecha a coordinar) para realizar una defensa oral, tanto del TPE como del examen escrito.